MTG Houdini

Architecture & Integration Reference

Magic: The Gathering Companion App

Next.js 16 · React 19 · Supabase · Clerk · Vercel

Generated:	April 17, 2026
Repository:	github.com/dlopez2392/MTG-app
Deployed at:	Vercel (auto-deploy, main branch)
Node.js:	"e 18.18 (v24 in production)

Table of Contents

1 — Application Overview

MTG Houdini · Magic: The Gathering Companion App

Purpose

MTG Houdini is a mobile-first Progressive Web App built with Next.js 16 and React 19. It combines real-time card lookup, collection management, deck building, price tracking, draft analytics, combo discovery, a life counter, a visual card scanner, and live MTG news — all in a single installable PWA hosted on Vercel.

Core Features

- Card Search — Full Scryfall integration with filters, autocomplete, and artwork lightbox
- Collection Manager — Binders backed by Supabase (cloud) + Dexie IndexedDB (offline/guest)
- Deck Builder — Editor with mana curve charts (Recharts), stats, and CSV export/import
- Card Detail — Prices (Scryfall, Card Kingdom, CardMarket, JustTCG), rulings, legality, combos, draft stats
- Card Scanner — Camera-based dHash visual matching against 49,191 pre-indexed artworks
- Life Counter — Multi-player counter with history log, persisted to IndexedDB
- News Feed — Aggregated RSS from 8 sources (WotC, TCGplayer, EDHREC, MTGGoldfish, etc.)
- Combos — Commander Spellbook API with 24-hour Supabase cache
- Draft Analytics — 17Lands per-card stats (ALSA, ATA, GIH WR, OH WR, IWD)
- Rules & Glossary — Offline-ready MTG comprehensive rules reference
- Wishlist — Cards to acquire, stored in IndexedDB

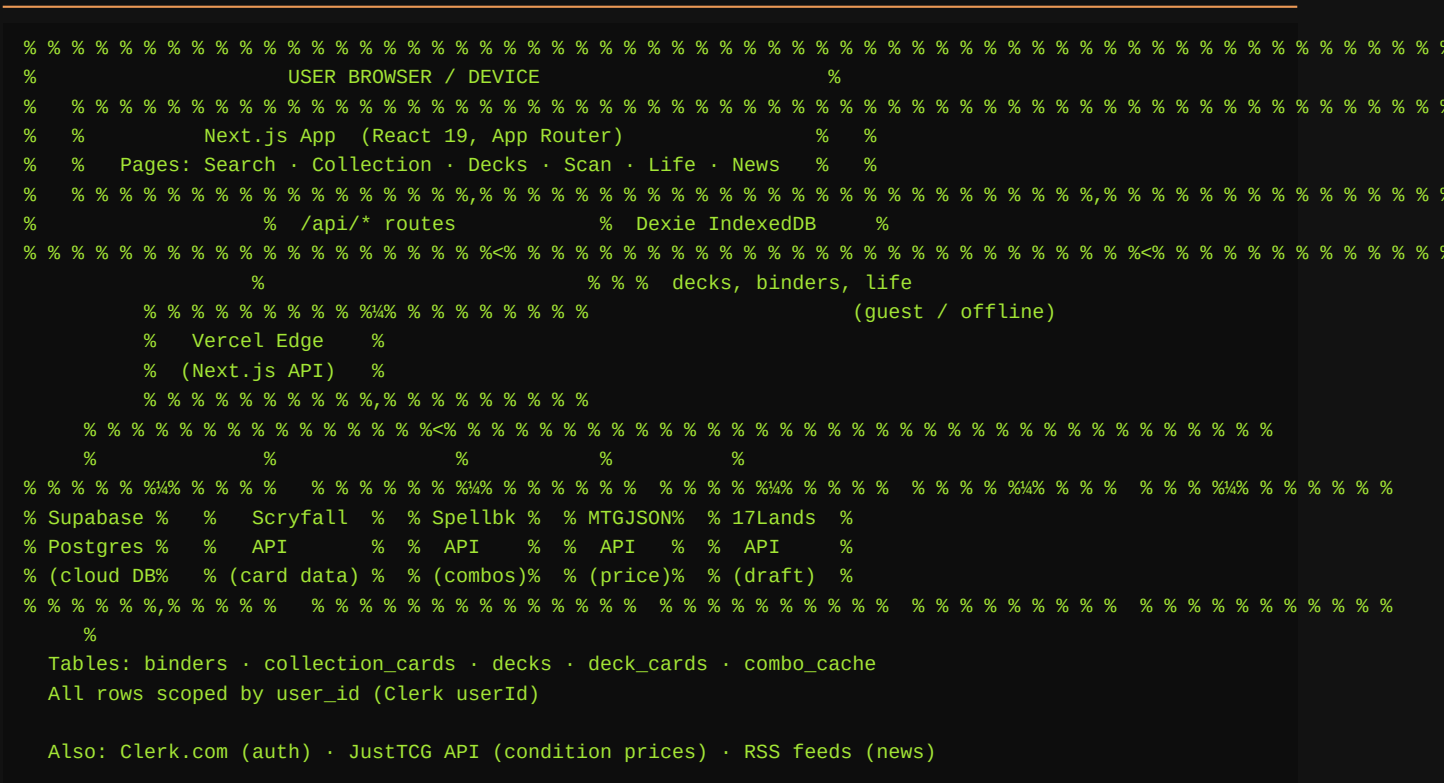
Tech Stack

Framework	Next.js 16.2.2 — App Router, Turbopack for local dev
UI Library	React 19.2.4
Styling	Tailwind CSS v4 — utility-first, dark theme throughout
Charts	Recharts 3 — mana curve bars, win-rate visualisation
Authentication	Clerk v7 (@clerk/nextjs) — sign-up, sign-in, JWT sessions
Cloud Database	Supabase (PostgreSQL) — collections, decks, combo cache
Local Database	Dexie v4 (IndexedDB wrapper) — guest mode & offline data
Image Processing	sharp v0.34 — server-side dHash computation for card scanner
OCR (fallback)	Tesseract.js v7 — client-side, lazy-loaded, used only when hash fails
Deployment	Vercel — auto-deploy from GitHub main branch
Language	TypeScript 5 throughout, strict mode enabled

2 — Architecture Overview

How all services connect to MTG Houdini

Service Map



Request Flow

- Browser !’ Next.js API Routes !’ External APIs (Scryfall, Spellbook, MTGJSON, 17Lands, JustTCG)
- Browser !’ Clerk SDK !’ Clerk.com (session validation, JWT tokens)
- Next.js API !’ Supabase (server-side Service Role Key — never exposed to client)
- Card Scanner !’ Canvas dHash (client) !’ POST /api/scan/search !’ in-memory index !’ Scryfall card fetch
- Guest users !’ Dexie IndexedDB only (no Supabase calls for collection/deck data)
- Signed-in users !’ Supabase via API routes (every row scoped by Clerk userId)

Caching Strategy

Scryfall search	next: { revalidate: 300 } — 5 min at Vercel edge
Scryfall card	next: { revalidate: 86400 } — 24 hours
MTGJSON set file	next: { revalidate: 86400 } — 24 hours (entire set JSON cached)
17Lands stats	next: { revalidate: 3600 } — 1 hour
JustTCG prices	next: { revalidate: 3600 } — 1 hour
Combo results	Supabase combo_cache table — 24 hours TTL (expires_at column)
Card hash index	Module-level memory cache on Vercel function instance (lives with process)

3 — Repository & Vercel Deployment

GitHub !’ CI/CD !’ Production

Repository

Provider	GitHub
URL	https://github.com/dlopez2392/MTG-app
Branch	main (single branch — push to main = deploy to production)
Language	TypeScript 5, strict mode
Build tool	Next.js built-in (next build) via Vercel

How Vercel Connects to GitHub

- Repository linked to Vercel via GitHub OAuth (authorised in Vercel dashboard)
- Every push to the main branch triggers an automatic production deployment
- Vercel webhook fires on push !’ installs dependencies !’ runs next build !’ deploys
- No separate staging or preview environments configured — all pushes go to production
- Deployment logs visible at: vercel.com !’ Project !’ Deployments

Deploy Pipeline

```
git push origin main
  %
  % % %º GitHub webhook !’ Vercel picks up commit
           %
           % % %º npm install  (all dependencies including sharp)
           % % %º next build   (TypeScript compile + route generation)
           % % %º Environment variables injected from Vercel dashboard
           % % %º Deploy to Vercel CDN !’ Production URL goes live (~2 min)
```

Vercel Project Settings

Framework Preset	Next.js (auto-detected)
Build Command	next build
Output Directory	.next
Install Command	npm install
Node.js Version	20.x (set in Vercel dashboard !’ Settings !’ General)
Root Directory	/ (repository root)
Production Branch	main

Static Assets

The file public/card-hashes.json (4.8 MB, 49,191 card artworks) is committed to the repository and served as a static file. It is loaded into server memory once per Vercel function instance and cached there for the lifetime of that process. No CDN or database needed for the hash index.

Manual Redeploy Commands

```
# Trigger redeploy without code changes:
git commit --allow-empty -m 'chore: trigger redeploy'
git push origin main
```

4 — Authentication — Clerk

Sign-up, sign-in, session management, and middleware

Overview

Authentication is handled entirely by Clerk (@clerk/nextjs v7). Clerk manages user accounts, sign-in/sign-up UI, session tokens, and OAuth providers. Authentication is optional — guests use the app fully with data stored locally; signed-in users get cloud sync via Supabase.

Integration Points

Root Layout (src/app/layout.tsx)

The entire app is wrapped in <ClerkProvider>. This makes Clerk hooks and session context available to every page and component in the app.

Middleware (src/middleware.ts)

```
import { clerkMiddleware } from '@clerk/nextjs/server';

export default clerkMiddleware();
// Runs on every request — validates session but does NOT block unauthenticated users.
// Auth gating happens per-route inside each API handler.
```

API Route Auth Pattern

```
import { auth } from '@clerk/nextjs/server';
import { getSupabase } from '@/lib/supabase/server';

export async function GET() {
  const { userId } = await auth();
  if (!userId) return Response.json({ error: 'Unauthorized' }, { status: 401 });

  // userId is the Clerk user ID — used as foreign key in ALL Supabase tables
  const sb = getSupabase();
  const { data } = await sb.from('binders').select('*').eq('user_id', userId);
  return Response.json(data);
}
```

Clerk Dashboard Configuration

Instance	Development (test keys — pk_test / sk_test prefix)
Sign-up methods	Email + password (configured in Clerk dashboard)
After sign-in URL	/ (NEXT_PUBLIC_CLERK_AFTER_SIGN_IN_URL)
After sign-up URL	/ (NEXT_PUBLIC_CLERK_AFTER_SIGN_UP_URL)
JWT template	Default Clerk JWT — no custom claims needed

User Identity in Supabase

Clerk's userId (format: 'user_2abc...') is stored as the user_id column in every Supabase table. There is no separate users table in Supabase — Clerk is the sole source of identity. All queries include .eq('user_id', userId) to scope data per user.

Environment Variables — Clerk

NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY	Public key — safe to expose to the browser
CLERK_SECRET_KEY	Server key — NEVER sent to client. API routes only.
NEXT_PUBLIC_CLERK_SIGN_IN_URL	Value: /sign-in
NEXT_PUBLIC_CLERK_SIGN_UP_URL	Value: /sign-up

5 — Database — Supabase (PostgreSQL)

Cloud persistence for signed-in users

Connection

Supabase is accessed exclusively from Next.js API Routes using the Service Role Key. The client is never initialised in browser code. Every API route calls getSupabase() which creates a server-side Supabase client with full service-role access.

```
// src/lib/supabase/server.ts
import { createClient } from '@supabase/supabase-js';

export function getSupabase() {
  return createClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.SUPABASE_SERVICE_ROLE_KEY!   // server-side only — full access
  );
}
```

Database Tables

binders

```
id              uuid PRIMARY KEY
user_id         text  -- Clerk userId (scopes all reads/writes)
name            text
description     text
cover_image_uri text
created_at      timestamptz
updated_at      timestamptz
```

collection_cards

```
id              uuid PRIMARY KEY
user_id         text, binder_id uuid REFERENCES binders
scryfall_id     text, name text, quantity int
set_code text, set_name text, collector_number text
image_uri text, price_usd text, type_line text, rarity text
is_foil boolean, created_at timestamptz
```

decks

```
id             uuid PRIMARY KEY
user_id        text, name text, format text
cover_card_id  text, cover_image_uri text
created_at     timestamptz, updated_at timestamptz
```

deck_cards

```
id           uuid PRIMARY KEY
user_id      text, deck_id uuid REFERENCES decks
scryfall_id  text, name text, quantity int
category     text  -- main | side | commander | maybe
mana_cost    text, type_line text, image_uri text
created_at   timestamptz
```

combo_cache

```
card_name   text PRIMARY KEY
combos      jsonb  -- array of EnrichedCombo objects
count       int
cached_at   timestamptz
expires_at  timestamptz  -- TTL: 24 hours from cached_at
```

Row-Level Security

Data is scoped per-user via .eq('user_id', userId) in every query inside the API routes. The Service Role Key bypasses Postgres RLS policies, so security is enforced at the application layer (Clerk userId from auth() call) rather than database-level RLS.

Environment Variables — Supabase

NEXT_PUBLIC_SUPABASE_URL	Public project URL — safe to expose
NEXT_PUBLIC_SUPABASE_ANON_KEY	Anon key — kept for reference, not actively used

SUPABASE_SERVICE_ROLE_KEY	Full-access server key — NEVER expose to client

6 — Local Storage — Dexie / IndexedDB

Offline-first guest mode

Purpose

Dexie v4 wraps the browser's built-in IndexedDB and provides the data layer for guest users (not signed in). All collection, deck, and life counter data is stored locally in the browser with no server calls. Signed-in users use Supabase; the switch is transparent to the UI.

Database Schema (src/lib/db/index.ts)

```
const db = new Dexie('mtg-houdini');

// v1 stores
decks:           ++id, name, format, folderId, createdAt
deckCards:       ++id, deckId, scryfallId, name, category
deckFolders:     ++id, name, parentId
binders:         ++id, name, createdAt
collectionCards: ++id, binderId, scryfallId, name
lifeGames:       ++id, createdAt

// v2 migration — added updatedAt index to decks
```

Data Routing

- Signed-in users !' all data in Supabase, accessible from any device
- Guest users !' data in browser IndexedDB only (lost if browser storage cleared)
- Life counter !' always uses IndexedDB regardless of auth state
- No automatic sync between local and cloud — manual import/export required to migrate

7 — Scryfall API Integration

Primary card data source — free, no API key required

Overview

Scryfall provides card data, artwork images, prices, rulings, and set information. All calls are proxied through Next.js API routes to add caching and avoid CORS. Scryfall's API is free with no authentication required.

API Routes

/api/scryfall/search	GET ?q=&page=&order=&dir=&unique=	Full Scryfall syntax search
/api/scryfall/named	GET ?fuzzy= or ?exact=	Single card by name
/api/scryfall/autocomplete	GET ?q=	Name completions (string[])
/api/scryfall/suggest	GET ?q=	Enriched suggestions (images + prices)
/api/scryfall/cards/[id]	GET	Single card by Scryfall UUID
/api/scryfall/sets	GET	All MTG sets
/api/scryfall/rulings/[id]	GET	Rulings for a card
/api/scryfall/image-proxy	GET ?url=	Proxy for artwork images (CORS)

Image Configuration

next.config.ts allows Next.js <Image> from cards.scryfall.io and svgs.scryfall.io. Image sizes used: small (146×204), normal (488×680), large (672×936), art_crop (artwork only).

Caching & Rate Limits

- Search results cached 5 min at Vercel edge (next: { revalidate: 300 })
- Card detail cached 24 hours (next: { revalidate: 86400 })
- Autocomplete debounced 300ms client-side to limit API calls
- Scryfall recommends max 10 requests/second — respected by hash builder (100ms batch delay)

8 — Commander Spellbook Integration

Combo lookup with 24h Supabase cache

Overview

Commander Spellbook provides a database of MTG combo recipes for the Commander format. The app queries their public REST API and caches results in Supabase for 24 hours.

Request Flow

- User opens card detail !' useCardCombos hook !' GET /api/combos?name={cardName}
- Server checks Supabase combo_cache WHERE card_name = ? AND expires_at > now()
- Cache HIT !' return cached combos immediately (no external call)
- Cache MISS !' fetch from commanderspellbook.com/variants/?q=card:"name"
- Filter: status in (OK, PREVIEW) and spoiler = false
- Map to EnrichedCombo shape (card images come from Spellbook directly)
- Write to combo_cache with expires_at = now() + 24h, return results

API Details

Base URL	https://backend.commanderspellbook.com/variants/
Auth	None required — public API
Cache TTL	24 hours in Supabase combo_cache table
Fallback	If exact match fails, retries with plain card name (no card: prefix)
Route	/api/combos?name=CARD_NAME

EnrichedCombo Shape

```
{
  id:          string;          // Spellbook variant ID
  cards: [{
    name:            string;
    zoneLocations:   string[];   // hand, battlefield, graveyard, etc.
    mustBeCommander: boolean;
    imageUri?:       string;
  }];
  produces:    string[];         // e.g. ['Infinite Damage', 'Infinite Mana']
  description: string;           // step-by-step instructions
  notes:       string;
  popularity:  number | null;
}
```

9 — MTGJSON Integration

Card Kingdom & CardMarket pricing

Overview

MTGJSON provides per-set JSON files with price history for Card Kingdom (retail + buylist) and CardMarket. These prices are not available in Scryfall. The entire set JSON is cached for 24 hours, so multiple card lookups within the same set are essentially free.

Endpoint

Route	/api/mtgjson/prices
Params	?set=SET_CODE&scryfallId=SCRYFALL_UUID
Source	https://mtgjson.com/api/v5/{SET}.json
Cache	next: { revalidate: 86400 } — full set JSON cached 24 hours
Auth	None required — public API

Response Shape

```
{
  cardKingdom: {
    retail:      string | null,   // e.g. '4.50'
    buylist:     string | null,
    retailFoil:  string | null,
    buylistFoil: string | null,
  },
  cardmarket: {
    retail:      string | null,
    retailFoil:  string | null,
  }
}
```

Lookup Logic

- Fetch https://mtgjson.com/api/v5/{SET}.json (e.g. DSK.json) — cached 24h
- Find card by matching identifiers.scryfallId === queried scryfallId
- Extract paper.cardkingdom and paper.cardkingdomFoil price maps
- Each price map: { '2025-01-15': '2.50', ... } — latest date used as current price
- Returns null for any price not available in that set's data

10 — 17Lands Integration

Draft analytics — ALSA, ATA, GIH WR, IWD

Overview

17Lands tracks Magic: The Gathering Arena draft data and publishes per-card statistics for each limited format. Stats appear on the card detail page under a 'Draft' tab. Data is CC BY 4.0 — attribution is shown in the UI.

Endpoint

Route	/api/17lands
Params	?set=SET_CODE&name=CARD_NAME
Source	https://www.17lands.com/card_ratings/data?expansion=SET&format=FORMAT
Cache	next: { revalidate: 3600 } — 1 hour
Format	Tried in order: PremierDraft !' QuickDraft !' TradDraft (first with data wins)
License	CC BY 4.0 — attribution shown in DraftStatsPanel component

Statistics

avgSeen (ALSA)	Average Last Seen At — how late in pack it wheels
avgPick (ATA)	Average Taken At — average pick position
winRate (GIH WR)	Games In Hand win rate — primary draft metric
openingHandWinRate (OH WR)	Win rate when card is in opening hand
drawnWinRate (GD WR)	Win rate when drawn after opening hand
everDrawnWinRate (GEV WR)	Win rate in games where card was ever drawn
neverDrawnWinRate (GNS WR)	Win rate in games where card was never drawn
drawnImprovementWinRate (IWD)	Impact of drawing the card on win rate
gameCount	Number of games in the sample set

Win Rate Colour Coding (UI)

- "e 58% — Green (excellent)
- "e 54% — Orange (above average)
- "e 50% — Yellow (average)
- < 50% — Red (below average)

11 — JustTCG Integration

Condition-specific TCGPlayer pricing

Overview

JustTCG provides condition-specific pricing (Near Mint, Lightly Played, etc.) sourced from TCGPlayer. Unlike Scryfall which gives a single market price, JustTCG breaks pricing down by condition. Requires a paid API key stored server-side.

Endpoint

Route	/api/justtcg/prices
Params	?name=CARD_NAME
Auth	X-API-Key header — uses JUSTTCG_API_KEY environment variable
Cache	next: { revalidate: 3600 } — 1 hour at Vercel edge
Key	JUSTTCG_API_KEY — set in Vercel environment variables (server-only)

Graceful Degradation

If JUSTTCG_API_KEY is not set, the endpoint returns { configured: false }. The UI falls back to Scryfall prices silently — no error is shown to the user.

12 — News Feeds (RSS)

8 MTG news sources aggregated client-side

Overview

The News page aggregates RSS feeds from 8 Magic publications. Parsing is done client-side in the browser on each page load — no server proxy required. Users can toggle which sources appear via Settings. Defined in src/lib/news/feeds.ts.

Sources

Magic: The Gathering (WotC)	magic.wizards.com/en/rss/news	— ON by default
TCGplayer Infinite	infinite.tcgplayer.com/rss	— ON by default
EDHREC	edhrec.com/articles/feed/	— ON by default
MTGGoldfish	mtggoldfish.com/news/rss	— ON by default
MTG Arena Zone	mtgazone.com/feed/	— ON by default
Hipsters of the Coast	hipstersofthecoast.com/feed/	— ON by default
Star City Games	articles.starcitygames.com/feed/	— OFF by default
ChannelFireball	channelfireball.com/feed/	— OFF by default

No Server-Side Caching

News feeds are fetched fresh on every page load client-side. No caching is configured because news content changes frequently and the sources' CDNs handle their own caching.

13 — Card Scanner (dHash Visual Fingerprinting)

49,191 artworks indexed — Delver Lens approach

Why dHash Instead of OCR

MTG uses stylised fonts that Tesseract.js misreads frequently. The dHash approach (used by Delver Lens) identifies cards by their artwork's visual fingerprint rather than text. It works with any card orientation or lighting as long as the artwork is visible.

Algorithm

```
1. Crop the artwork region from camera frame:
   artX = 7%  artY = 14%  artW = 86%  artH = 43%  (of video dimensions)

2. Resize crop to 9 × 8 pixels, convert to greyscale

3. For each of 8 rows, compare each pixel to its right neighbour:
   bit = 1 if left pixel is brighter, 0 otherwise
   !' 64 bits total, encoded as 16-character hex string
   !' Example: 'a3f1b2e4c9d07851'

4. Compute Hamming distance to every entry in the hash index:
   distance "d 10  !' strong match  !' show card immediately
   distance 11-20 !' weak match    !' show picker (top 5 candidates)
   distance > 20  !' no match      !' fall back to OCR (Tesseract.js)
```

Key Files

src/lib/scan/dhash.ts	Shared dHash math — dHashFromRaw, hammingDistance, findMatches
src/app/api/scan/search/route.ts	POST endpoint — resizes with sharp, hashes, searches index
scripts/build-hash-index.mjs	One-time CLI builder — downloads Scryfall bulk, hashes all artworks
public/card-hashes.json	Pre-built index (4.8 MB, 49,191 unique artworks — committed to repo)
src/hooks/useCameraScanner.ts	React hook — camera, client-side dHash, API calls, OCR fallback
src/components/scan/ScanPageClient.tsx	Camera UI — guide overlay, scan list, matched card panel

Index File Format

```
{
  version:   1,
  generated: '2026-04-17T02:14:39.002Z',
  count:     49191,
  cards: [
    { id: 'scryfall-uuid', n: 'Card Name', s: 'set', h: 'a3f1b2e4c9d07851' },
    ...
  ]
}
```

OCR Fallback

If the hash index is absent or no match within distance 20 is found, Tesseract.js is lazy-loaded (~4 MB download on first use) and runs OCR on a 4× upscaled greyscale crop of the card's name bar region. The result is sent to Scryfall's fuzzy name API.

14 — Environment Variables

All required keys, where to set them, and their purpose

Complete Variable Reference

NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY

' Public Set in: Vercel dashboard + .env.local

Clerk public key — initialises Clerk's browser SDK. Safe to expose.

CLERK_SECRET_KEY

& SECRET Set in: Vercel dashboard ONLY

Clerk server key — validates sessions in API routes. NEVER expose to browser.

NEXT_PUBLIC_CLERK_SIGN_IN_URL

' Public Set in: .env.local

Path to sign-in page. Value: /sign-in

NEXT_PUBLIC_CLERK_SIGN_UP_URL

' Public Set in: .env.local

Path to sign-up page. Value: /sign-up

NEXT_PUBLIC_SUPABASE_URL

' Public Set in: Vercel dashboard + .env.local

Supabase project URL. Format: https://{ref}.supabase.co — safe to expose.

NEXT_PUBLIC_SUPABASE_ANON_KEY

' Public Set in: Vercel dashboard + .env.local

Supabase anon key — kept for reference, not actively used in app code.

SUPABASE_SERVICE_ROLE_KEY

& SECRET Set in: Vercel dashboard ONLY

Full Supabase access. Used server-side only. NEVER expose to client.

NEXT_PUBLIC_FORMSPREE_URL

' Public Set in: Vercel dashboard + .env.local

Formspree endpoint for the feedback form in Settings.

JUSTTCG_API_KEY

& SECRET Set in: Vercel dashboard ONLY

JustTCG pricing API key. Server-only. App degrades gracefully without it.

Setting Variables on Vercel

vercel.com !' Your Project !' Settings !' Environment Variables. Add each variable, select environments (Production, Preview, Development), and save. Secret variables (marked &) should only exist in Vercel — never committed to the repo.

Local Development (.env.local)

The .env.local file in the project root is loaded automatically by Next.js during development. It is listed in .gitignore and must NEVER be committed to the repository.

15 — Maintenance Schedule

What needs running, when, and what maintains itself

New Set Release (Every ~3 Months)

When Wizards releases a new MTG set, the card hash index must be updated so the scanner recognises the new artworks. This is the only recurring manual maintenance task.

```
# Step 1 — Add new set artworks to the index (takes ~2 minutes)
cd C:/Users/danlo/MTG-app
SET_CODE=xxx RESUME=true node scripts/build-hash-index.mjs
# Replace xxx with the set code, e.g.: fdn  dsk  mh3  blb

# Step 2 — Commit and push (Vercel auto-deploys in ~2 minutes)
git add public/card-hashes.json
git commit -m 'feat: add [SET NAME] artworks to hash index'
git push origin main
```

Full Index Rebuild (Only If Needed)

Only required if the index file is lost, corrupted, or you want a clean rebuild from scratch.

```
# Full rebuild — ~45-60 minutes, downloads all ~200MB of Scryfall bulk data
node scripts/build-hash-index.mjs

# Resume a previously interrupted build
RESUME=true node scripts/build-hash-index.mjs
```

Everything Else is Automatic

- Card prices (Scryfall) — fetched live, cached 24h at Vercel edge
- Card Kingdom prices (MTGJSON) — fetched live per set, cached 24h
- JustTCG prices — fetched live, cached 1h
- 17Lands draft stats — fetched live, cached 1h. Always current.
- Commander Spellbook combos — Supabase TTL cache auto-refreshes on expiry
- News feeds — client-side fetch on each page load
- Card search / autocomplete — proxied live to Scryfall (5 min cache)

- Card images — served from Scryfall CDN, no local copies needed
- Clerk auth — managed SaaS, no maintenance needed
- Supabase database — managed PostgreSQL, auto-backups by Supabase
- Vercel deployment — every push to main triggers auto-deploy
- combo_cache table — self-cleaning: expired rows are simply never queried

Troubleshooting Quick Reference

Scanner not recognising new cards	Run SET_CODE=xxx RESUME=true node scripts/build-hash-index.mjs
public/card-hashes.json missing	Run full rebuild: node scripts/build-hash-index.mjs
Prices not loading	Check Vercel function logs for MTGJSON / Scryfall errors
Combos not showing	Check Supabase combo_cache — may have stale data from API changes
Login not working	Check Clerk dashboard for suspended accounts or key issues
Build failing on Vercel	Check Vercel deploy logs — usually a TypeScript error in recent commit
Scanner falls back to OCR always	Verify public/card-hashes.json is in repo and > 4 MB

Quick Reference Card

Services & URLs

GitHub Repo	github.com/dlopez2392/MTG-app
Vercel App	vercel.com/dlopez2392/MTG-app (Deployments + Logs + Env Vars)
Clerk	clerk.com — user accounts, auth config
Supabase	supabase.com — database tables, SQL editor, backups
JustTCG	justtcg.com — API key management

All API Routes

Card search	GET /api/scryfall/search?q=&page=&order=
Card by ID	GET /api/scryfall/cards/[id]
Card by name	GET /api/scryfall/named?fuzzy=
Autocomplete	GET /api/scryfall/autocomplete?q=
Combos	GET /api/combos?name=
CK prices	GET /api/mtgjson/prices?set=&scryfallId=
Draft stats	GET /api/17lands?set=&name=
JustTCG	GET /api/justtcg/prices?name=
Hash scanner	POST /api/scan/search body: { image: '...' }

Critical Files

Root layout	src/app/layout.tsx — ClerkProvider wrapper
Auth middleware	src/middleware.ts — clerkMiddleware()
Supabase client	src/lib/supabase/server.ts — getSupabase() with service role
IndexedDB schema	src/lib/db/index.ts — Dexie table definitions
dHash utilities	src/lib/scan/dhash.ts — hammingDistance, findMatches
Hash index	public/card-hashes.json — 49k artworks, 4.8 MB
Index builder	scripts/build-hash-index.mjs — run when new sets release

Secret Environment Variables (Vercel only — never commit)

```
CLERK_SECRET_KEY            # Clerk server-side auth
SUPABASE_SERVICE_ROLE_KEY   # Full Supabase DB access
JUSTTCG_API_KEY             # JustTCG pricing API
```

Hash Index Maintenance

```
# New set (run every ~3 months when sets release):
SET_CODE=xxx RESUME=true node scripts/build-hash-index.mjs
git add public/card-hashes.json && git commit -m 'update index' && git push
```